



ICS3U Resource Book

Revised August 2013

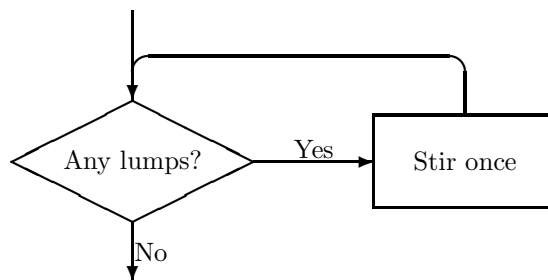
4.1 while Statements

In cooking, a recipe may tell us that, as long as there are lumps in a sauce, we should stir it. We can make these instructions more Java-like by writing them in the form

```
while (there are lumps)
    give sauce a stir;
```

The idea here is that we repeatedly check for lumps and, if we find any, stir the sauce to try to get rid of them.

We can illustrate this process as we did with an `if` statement, by using a *flow chart*. Here is a flow chart for our cooking process. Perhaps you can see from the diagram why we call this structure a *loop*. As long as there is lumpiness, we follow the path that carries us to stirring and then back to testing for lumps.

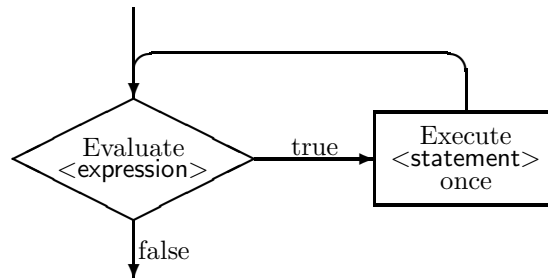


The general form of Java's `while` statement follows the same pattern as the Java-like form we used for our cooking analogy:

```
while (<boolean expression>)
    <statement>
```

Execution of a `while` statement is similar to execution of an `if` statement that has no `else` clause. The `<boolean expression>` (which must be inside parentheses) is evaluated and, if it is false, then control passes to the statement after the `while`. If, on the other hand, the `<boolean expression>` is true, then the `<statement>`, known as the *body* of the loop, is executed once. Unlike the `if` statement, the `while` statement repeats the process. The `<boolean expression>` is evaluated again and, if it is still true, the `<statement>` is executed one more time. This is repeated until the `<boolean expression>` is false, at which time control passes to the statement after the `while`.

A flow chart for a general **while** statement can be used to illustrate this process.



The next example shows the use of a **while** statement in a complete program.

Example 1

The following program reads integers and prints them along with their squares. It continues to read and print until the user provides a value of zero.

```

class PrintSquares
{
    public static void main (String[] args)
    {
        System.out.println("Give an integer (zero to stop)");
        int value = In.getInt();
        while (value != 0)
        {
            System.out.println(value + "    " + value*value);
            System.out.println("Next integer (zero to stop)");
            value = In.getInt();
        }
    }
}
  
```

There are a number of points that we should note in the use of a `while` statement in this program.

1. The statement `int value = In.getInt();` that precedes the `while` statement is necessary to *initialize* the variable `value` before the loop is entered for the first time. Without this initialization, `value` would be undefined the first time that the `while` statement was encountered and an error would occur. This pattern of initialization followed by a loop is one that we will see many times.
2. By using a compound statement with brace brackets, we can have more than one statement in the body of the loop, exactly as we did with `if` statements.
3. Because the statement(s) within the body of the loop will be executed repeatedly as long as the condition that controls the loop is true, there must be some action within the loop that will eventually cause the condition to become false. In the example, this is accomplished by reading a value of zero. We say that zero acts as a *sentinel* to halt the loop.
4. If the boolean expression that controls the loop is false the first time that the `while` statement is encountered, then the statement(s) inside the loop will never be performed. Control will pass directly to the statement following the `while`. In the example, this would occur if the first value that a user entered were zero, the sentinel value.
5. If the expression that controls the loop is never changed to false, then the program will get stuck in the `while`, executing it for as long as the program is allowed to run. If this happens (and it will happen to you, sooner or later), we say that the program is caught in an *infinite loop*. The technique for stopping a program caught in an infinite loop varies from system to system. There is usually some combination of keys that will do the trick; your system supervisor should be able to tell you what to do.
6. If a statement in a `while` loop changes the boolean expression that controls the loop to false, the program does not immediately proceed to the statement following the loop. Once a program has entered the body of a `while` loop, the entire sequence of statements in the body of the loop is executed and only then is the boolean expression re-evaluated to see if the loop should be entered again.

7. Once the loop has been exited, the boolean expression that controls it *must* be false. (Otherwise we would never have left the loop.)

In many applications we want to find the total amount of some quantity. Loops enable us to do this easily and efficiently.

Example 2

The following program uses a loop to sum a set of values and find their average.

```
class AddMarks
{
    public static void main (String[] args)
    {
        /* This program reads the marks obtained for a      */
        /* student on an exam and finds the average mark. */
        /* It prints the average mark, rounded to the      */
        /* nearest integer.                                */

        System.out.println("Submit marks (value < 0 to stop)");
        int totalMarks = 0;
        int numberOfMarks = 0;
        int nextMark = In.getInt();
        while (nextMark >= 0)
        {
            totalMarks += nextMark;
            numberOfMarks++;
            System.out.println("Next mark:");
            nextMark = In.getInt();
        }
        if (numberOfMarks > 0)
        {
            int average =
                Math.round((float)totalMarks/numberOfMarks);
            System.out.println("Average for " + numberOfMarks
                               + " students is " + average);
        }
    }
}
```

In this program, the initialization includes both setting the variables `totalMarks` and `numberOfMarks` to zero as well as reading the initial value of `nextMark`. As in the previous example, the statement

```
nextMark = In.getInt();
```

is repeated inside the loop to read subsequent values. In this way, the program will eventually read a sentinel value (here, any negative number) and conclude by finding the average. Notice also that the program only finds the average if there has been at least one mark entered; if only a sentinel was supplied, the program does not attempt to calculate an average.

Exercises 4.1

1. What is the minimum number of times that the body of a `while` statement can be executed?
2. What is the maximum number of times that the body of a `while` statement can be executed?
3. What will be printed by this fragment?

```
int m = 10;
int n = 0;
while (m > n)
{
    System.out.println(m + " " + n);
    m--;
    n += 2;
}
```

4. Write a program that reads a positive integer and then finds the smallest power of two that is greater than or equal to the number that was read. For example, if the program reads the value 25, it should note that $32 = 2^5$ is the smallest power of two greater than or equal to 25.
5. Write a program that prompts the user for a sequence of integers, using zero as a sentinel. The program should count the number of times that consecutive values are equal. For example, if the input is


```
3  6  7  7  4  4  4  6  0
```

 then the program should determine that there are three cases in which consecutive values are equal.

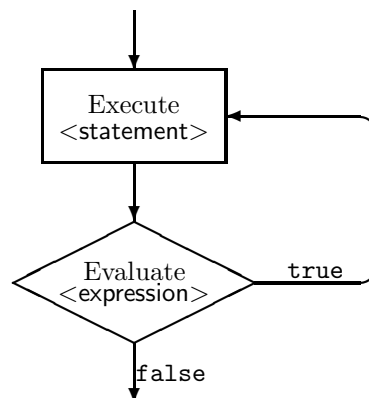
4.2 do Statements

In the last section, we suggested that a **while** loop was Java's way of implementing commands of the form "as long as there are lumps in a sauce, we should stir it". If we rearrange this sentence a bit, we can rephrase it as "stir the sauce as long as there are lumps in it". The difference between the two statements is subtle but it is often important. In the first case, we repeatedly check for lumps first and then, if there are any, we stir. In the second case we repeatedly stir and then check for lumps. For these two kinds of repetitive instructions, Java has two kinds of loops. As we saw in the last section, the **while** is appropriate for the first kind of situation. To handle the second we have the **do** statement. The general form of the **do** statement is:

```
do
    <statement>
while (<boolean expression>);
```

To execute a **do** statement, Java first executes the **<statement>** and then evaluates the **<boolean expression>** which must, as usual, be enclosed by parentheses. If the expression is true, the process is repeated.

A flow chart for a **do** statement takes the following form.



Since a `do` statement executes the statement in its body before evaluating the boolean expression that controls the loop, it is useful in situations where we know that we want to perform the loop at least once. Reading input and repeatedly checking to see that it is valid is a common example of this kind of situation, as the next example illustrates.

Example 1

This fragment forces a user to supply a value that lies in the range from one to ten.

```
int value;
do
{
    System.out.println("Give an integer from one to ten");
    value = In.getInt();
}
while (value < 1 || value > 10);
```

Notice in the example that the condition that controls the loop is the negation of what we want to happen. We want to get a value from one to ten so we force the user to stay in the loop as long as the user is providing values that are *not* in this interval.

Exercises 4.2

1. For what kind of looping situations is a `do` usually preferable to a `while`?
2. What would be written by the program fragment shown below if it were given the following input sequence?

```
-5
0
26
```

```
System.out.println("How many items to add?");
do
{
```

```
    howMany = In.getInt();
    if (howMany <= 0)
        System.out.println("Give a positive value.");
}
while (howMany <= 0);
```

3. Write a fragment that forces a user to supply either `y` or `n` in response to the question "Continue? Respond with y or n".
4. Write a program fragment that asks the user to supply a letter of the alphabet, repeatedly rejecting responses until the user gives either an upper case or a lower case letter of the alphabet.
5. Write a program that first forces the user to supply a positive integer and then prints the number and the sum of its digits.

4.3 Simple for Statements

Java's final loop statement is the `for` statement. Returning one last time to our cooking analogy, we might have instructions for eliminating lumpiness phrased in the form "stir mixture for 100 strokes". Here we are not asked to check on the state of the mixture before or after each stroke — we just keep stirring until we have done one hundred repetitions. We can write these instructions in a Java-like form as follows:

```
int count;
for (count = 0; count < 100; count++)
    give sauce a stir;
```

To see exactly what this means, we can rewrite it using an equivalent fragment that uses a `while` format, as follows:

```
int count = 0;
while (count < 100)
{
    give sauce a stir;
    count++;
}
```

By comparing the `for` and `while` forms, we can see that the first expression inside the parentheses of the `for` is an initializer, the second

expression (which must be of type `boolean`) is a condition that controls the loop, and the third expression is a modifier that is evaluated after each execution of the body of the loop. The next example shows how an actual Java `for` statement works. The example uses a `for` loop to find the value of the sum of a series.

Example 1

The following fragment finds the sum of the series $1^2 + 2^2 + \dots + 100^2$. In the fragment, the variables `i` and `sum` are both of type `int`.

```
sum = 0;
for (i = 1; i <= 100; i++)
    sum += i*i;
```

This `for` statement first initializes `i` to 1. Then it repeatedly checks that $i \leq 100$ and, if it is, it increments `sum` by `i*i` and increments `i` by 1.

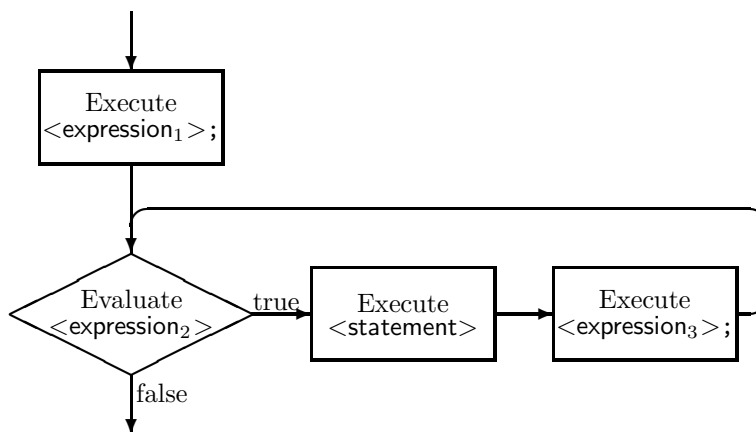
The general form of Java's `for` statement follows the form seen in the example. The syntax of a `for` statement is given below.

```
for (<expression1>; <expression2>; <expression3>)
    <statement>
```

As we did with our cooking example, we can write a general `for` statement using a `while` statement. The form of the `for` statement shown above is exactly equivalent to the following fragment.

```
<expression1>;
while (<expression2>)
{
    <statement>
    <expression3>;
}
```

Although the `for` and the `while` are equivalent, the format of the `for` often makes the loop's action much clearer to the reader. The structure of a `for` statement and its relationship to a `while` statement may be made clearer by studying its flow chart.



Often, `for` statements are controlled by counters (as we saw in Example 1 where the variable `i` controlled the loop). Since we often need such loop control variables, Java allows us to declare them right in the `for` statement as the next example shows.

Example 2

The program fragment shown here prints values of the function $y = x^3 - 3$ for x taking values $-6, -4, \dots, 6$.

```
for (int x = -6; x <= 6; x += 2)
    System.out.println(x + "    " + (x*x*x - 3));
```

Variables declared in the initialization part of a `for` statement are only usable inside that statement. Thus, in Example 2, the variable `x` is only usable inside the `for`. If we added the statement

```
System.out.println(x);
```

after the `for`, we would get an error message because a variable declared within a statement is only known inside that statement.

Exercises 4.3

1. What does each statement print?

```
(a) for (int i = -3; i <= 3; i++)
    System.out.print("*");
```

- (b)

```
for (int countDown = 5; countDown > 0; countDown--)
    System.out.println(countDown + " seconds");
```
- (c)

```
for (char letter = 'P'; letter <= 'S'; letter++)
    System.out.println("Give me a " + letter);
```
- (d)

```
for (int i = 2; i < 100; i *= i)
    System.out.println(i);
```

2. Write statements that will print a table of values of the function $f(x) = 2x + 5$ for the indicated values of x .

- (a) $x = 6, 5, 4, \dots, 0$
- (b) $x = 0, 3, 6, \dots, 30$
- (c) $x = -15, -10, -5, \dots, 15$
- (d) $x = 1, 2, 4, 8, \dots, 1024$

3. Write a fragment that uses a **for** statement to perform the indicated action.

- (a) Set the **double** variable **sum** to the value of

$$\frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{1000}$$

- (b) Set the **double** variable **sum** to the value of

$$\sqrt{100} + \sqrt{200} + \sqrt{300} + \dots + \sqrt{5000}$$

- (c) Set the **long** variable **product** to the value of

$$1 \times 2 \times 3 \times \dots \times 20$$

- (d) Set the **int** variable **total** to the value of

$$(-12)^3 + (-11)^3 + (-10)^3 + \dots + (20)^3$$

- (e) Set the **double** variable **sum** to the value of

$$1 + \sqrt{2} + \sqrt[3]{3} + \dots + \sqrt[25]{25}$$

4. Suppose that a large piece of paper with an area of 1.0 m^2 and a thickness of 0.090 mm is cut in half and the two pieces are stacked, one on top of the other. Suppose further that the process of cutting in half and stacking is repeated over and over again. Write a program to find both the thickness of the pile and the area of each piece after the procedure has been carried out forty times.

5. Write a program that reads a positive integer n and then prints an “ n -times table” containing values up to $n \times n$. For example, if the program reads the value 5, it should print

5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25

Assume that the input is valid.

6. (a) Write a program that will print a table containing the integers from one to forty along with their squares, square roots, and reciprocals.
- (b) Modify your program so that it inserts a blank line after every fifth line of the table.

4.4 Variations on for Statements

Although most uses of `for` statements have the simple form that we saw in the last section, many variations are possible. One of the ways in which we can extend the statement is by using commas to combine multiple expressions into larger ones as the next example illustrates.

Example 1

The fragment shown below demonstrates the use of commas to build complex expressions from simpler ones in `for` statements.

```
for (i = 0, j = 5; i < j; i++, j--)
    System.out.println(i + " " + j);
```

The output from this fragment will be

0 5
1 4
2 3

The initialization here consists of the assignments `i = 0` and `j = 5` while the modification after each repetition of the loop consists of both an increment of `i` and a decrement of `j`.

There is no limit to the number of expressions that can be included in the first and third parts of a `for` statement. If we want more than two, we can simply add others using more commas. We cannot do this with the second expression; it must be a single boolean expression.

If we carry the process of placing multiple expressions in the header of the `for` to its extreme, we can sometimes produce a loop that has nothing but an empty statement in its body. Such a situation is shown in the next example.

Example 2

This fragment finds and prints the sum of the squares of the numbers from one to ten.

```
for (i = 1, sum = 0; i <= 10; sum += i*i, i++)  
;  
System.out.println("Sum of squares is " + sum);
```

Here the body of the `for` consists only of an empty statement. We have placed the empty statement's semi-colon on a line by itself so that it is clearly visible to a reader.

Instead of increasing the number of expressions in the first and third parts of a `for` statement, we can eliminate either or both of them completely if there is no need for explicit initialization or modification.

Example 3

The statement shown below produces a loop that will repeatedly print its message as long as the program is allowed to run.

```
for ( ; true ; )  
    System.out.println("Help! I'm in an infinite loop!");
```

Notice that, although the first and third expressions are empty, the semi-colons that are used as separators must still be included.

Exercises 4.4

1. Assuming that all variables are of type `int`, determine the output of each fragment.

(a) `for (i = 1, j = 12; i < j; i++, j -= 2)`

`System.out.println(i + " " + j);`

(b) `for (i = 1, product = 1; i <= 5; product *= i, i += 2)`

`System.out.println(product);`

(c) `for (n=5, i=1; n>1; System.out.println(i), n--, i*=2)`

`;`

2. Rewrite each fragment as a single `for` statement whose body consists of only an empty statement (as illustrated in Example 2).

(a) `product = 1;`

`for (int i = 10; i > 0; i--)`

`product *= i;`

(b) `max = i = 0;`

`j = 10;`

`for (; i < 10 ;)`

`if (i * j > max)`

`max = i++ * j--;`

3. Write a `for` statement that has the form of the `for` statement in Example 2 and that performs the indicated action.

(a) Set the `double` variable `sum` to the value of

$$\frac{1}{2} + \frac{2}{3} + \frac{3}{4} + \cdots + \frac{19}{20}$$

(b) Set the `int` variable `total` to the value of

$$1 \times 100 + 2 \times 98 + 3 \times 96 + \cdots + 50 \times 2$$

4.5 Comparing Loop Structures

With three loop structures available in Java, how do we decide which to use in a given situation? To some extent, this is a matter of personal taste, but usually one of the statements is more appropriate than either of the others.

The **for** statement should usually be used if we know, before we start the loop, how many repetitions we want. Problems that call for the reading of a given number of data items or the production of a table of a given size are examples of situations where we would normally use a **for** statement.

The **while** and **do** statements should normally be used when the number of repetitions is not known before we start execution of the loop. If it is convenient to test the condition that controls the loop before entry to the loop body, then a **while** is the usual choice. If the condition is only established within the body, then a **do** should be used. If it is possible that the loop body may never be entered, a **while** statement should be used; if it is certain that the loop body will be executed at least once, then a **do** might be the better choice.

Example 1

All three program fragments shown here have the same effect. They all find the sum of the series

$$1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5}$$

In each part, the variable **sum** is of type **double** while **i** is of type **int**.

```
(a) sum = 0;
    i = 1;
    while (i <= 5)
    {
        sum += 1.0/i;
        i++;
    }
```

```
(b) sum = i = 0;
    do
    {
        i++;
```

```
        sum += 1.0/i;
    }
    while (i < 5);
(c) for (sum = 0, i = 1; i <= 5; i++)
    sum += 1.0/i;
```

The solution using a `for` statement is the shortest and clearest.

We have suggested that counted loops use `for` statements while conditional loops use `while` or `do` statements. Sometimes, however, loops have both of these elements. For these situations, the choice of structure is very often a matter of personal preference.

Example 2

As part of a program to generate report cards, one segment is required to read and total the marks. Most students take `MAX_SUBJECTS` subjects and input for them consists of that many marks. Some students take fewer subjects. Input for them consists of their marks followed by a sentinel value of `-1`.

In this problem, the number of repetitions of the loop is sometimes simply counted (for students taking the maximum number of subjects) but sometimes conditional (for students with fewer subjects). The segment shown below will give the correct values for `totalMarks` and `numberOfMarks` for input of either form. We have solved the problem with a `do` but other solutions are possible.

```
totalMarks = numberOfMarks = 0;
do
{
    System.out.println("Next mark please");
    mark = In.getInt();
    if (mark != -1)
    {
        totalMarks += mark;
        numberOfMarks++;
    }
}
while (mark != -1 && numberOfMarks < MAX_SUBJECTS);
```

Exercises 4.5

1. Describe what the fragment does and then rewrite it using a `for` statement.

(a)

```
count = 1;
while (count < 10)
{
    System.out.println(count+" "+Math.sqrt(count));
    count++;
}
```

(b)

```
i = 5;
do
{
    i--;
    System.out.println(Math.pow(i,3));
}
while (i > 0);
```

2. Describe what the fragment does and then rewrite it using a `while` statement.

(a)

```
for (i = 0; i < 10; i++)
{
    x = 2 * i - 10;
    y = Math.abs(x / 5 - 2);
    System.out.println(x + " " + y);
}
```

(b)

```
product = 1;
do
{
    System.out.println("Next value please");
    next = In.getInt();
    if (next != TERMINATOR)
        product *= next;
}
while (next != TERMINATOR);
```

3. Describe what the fragment does and then rewrite it using a `do` statement.

(a)

```
System.out.println("Enter a value - zero to stop");
value = Math.abs(In.getInt());
biggest = value;
while (value != 0)
{
    if (value > biggest)
        biggest = value;
    System.out.println("Enter a value (0 to stop)");
    value = Math.abs(In.getInt());
}
```

(b)

```
for (i = 0; i == 0; )
{
    System.out.println("Continue? (Y/N)");
    response = In.getChar();
    if (response == 'Y' || response == 'N')
        i = 1;
}
```

4. State, with reasons, which loop structure you think would be most appropriate for solving each problem. Do *not* actually solve the problem.

- (a) Find the sum of the cubes of the numbers from 1 to 100.
- (b) Repeatedly prompt a user for a password, rejecting any submissions until the correct password has been provided.
- (c) Read and sum positive integers until a sentinel value of -1 is read.
- (d) Determine the number of times that a positive integer can be divided by two.
- (e) Find the amount to which \$1 will grow in ten years at a rate of 8%.

4.6 Nesting Loop Structures

In using a loop, the body can contain any valid Java statements, including other loop statements. Thus we can have **while**, **do**, and **for** statements within other **while**, **do**, and **for** statements. If a first loop contains a second loop, we say that the second is *nested* within the first. The first loop is referred to as the *outer loop* and the second as the *inner loop*.

To take a simple example, suppose that we want to print a triangular pattern of asterisks, like the one shown below.

```
*
**
***
****
*****
```

The statement `System.out.print("*");` will print one asterisk. To print a complete line of asterisks, we can use a loop containing this statement, with the loop followed by the statement `System.out.println();` to end the line. To print a number of such lines, we can use another loop containing the instructions for writing one line. The result is shown in the next example.

Example 1

The following fragment prints a triangle of asterisks with the size of the triangle determined by the value of `numberOfRows`.

```
for (row = 1; row <= numberOfRows; row++)
{
    for (position = 1; position <= row; position++)
        System.out.print("*");
    System.out.println();
}
```

As we have already noted, loop nesting is not limited to **for** statements. We can nest any form of loop inside any other form, as illustrated in the next example.

Example 2

Suppose that we want to write a program that repeatedly prompts a user to provide integer values (until the user provides a value of zero). For each non-zero value read, the program prints the number and the sum of its digits. We can begin to attack the problem by constructing code to handle *one* value. Our strategy is to first eliminate any negative sign from the number and then strip off the final digits, one by one, and add them up. This continues until there are no more digits to strip off. A `while` loop handles this nicely. In the fragment, `n` is the value whose digits are to be added.

```
int absN = Math.abs(n);
int digitSum = 0;
while (absN > 0)
{
    digitSum += absN % 10; // add last digit to digit sum
    absN /= 10;           // and strip it off the number
}
System.out.println("Sum of digits of "+n+" is "+digitSum);
```

Having solved the problem for one value of `n`, we can then embed this into a loop that keeps prompting and reading until the user supplies a value of zero. We have chosen to use a `do` statement for this loop. Here is the final program.

```
class DigitAddition
{
    public static void main (String[] args)
    {
        int n;
        do
        {
            System.out.println("Give an integer (0 to stop)");
            n = In.getInt();
            if (n != 0)
            {
                int absN = Math.abs(n);
                int digitSum = 0;
                while (absN > 0)
                {
```

```

        digitSum += absN % 10; // add last digit
        absN /= 10;           // and delete it
    }
    System.out.println("Sum of digits of " + n
        + " is " + digitSum);
}
}
while (n != 0);
}
}

```

Exercises 4.6

1. Assuming that `SIZE` has the value 5, determine the output that will be produced by the following fragment.

```

for (i = 1; i <= SIZE; i++)
{
    for (j = SIZE; j >= i; j--)
        System.out.print("*");
    System.out.println();
}

```

2. Write a program that will print a triangular pattern of asterisks whose maximum width is determined by input from the user. The diagram shows the pattern that should be printed if the user's input is 5.

```

*
**
***
****
*****
****
***
**
*

```

3. Write a program using nested loops that prints the following pattern.

```
1
12
123
1234
12345
```

4. Write a program using nested loops that prints the following pattern.

```
54321
 4321
   321
    21
     1
```

5. Three positive integers a , b , and c with $a < b < c$ form a *Pythagorean triplet* if $a^2 + b^2 = c^2$. For example, 3, 4, and 5 form a Pythagorean triplet since $3^2 + 4^2 = 5^2$. Write a program that first prompts the user for a positive integer and then finds and prints all Pythagorean triplets whose largest member is less than or equal to that integer.
6. Write a program that will first prompt the user for the height and width of a pattern and will then print a pattern that is the given number of lines high and the given number of characters wide. The pattern should contain stars and blanks similar to the pattern shown in the flag of the United States. For example, if the user specified a height of 5 and a width of 11, the program should produce the following output. Assume that the user provides valid input.

```
* * * * *
 * * * * *
* * * * *
 * * * * *
* * * * *
```

7. Write a program that first prompts the user (repeatedly, if necessary) to supply a single digit (0, 1, 2, ..., 9). The program should then repeatedly prompt the user to supply integer values until the user provides a value of zero which acts as a sentinel to terminate the program. For each integer read (except for the final zero), the program should count the number of occurrences of the digit read in the first part of the program and should print this count.

4.7 Avoiding Errors and Debugging

Avoiding Errors

1. In setting up a loop, take the time to choose an appropriate structure. If a simple counted loop is required and you know how many times the loop will be performed, use a `for` statement. If a conditional loop is called for, you have to choose between a `while` or a `do`. To make this decision, it is often helpful to use the following criteria.
 - (a) If it is easier to test the condition that controls the loop at the start, you should probably use a `while` but if it is easier to test at the end of the loop, use a `do`.
 - (b) If it is possible that the loop might be executed zero times, use a `while` but if it is certain that it will be executed at least once, a `do` might be more appropriate.If a loop has both counting and conditional elements, it is usually (but not always) best to use a `for` statement.
2. Enter a loop appropriately. Be sure that you have correctly initialized all variables that need to be defined before you enter the loop.
3. In a conditional loop, be sure that there is some action that ensures that the program will eventually exit from the loop.
4. Be careful, as usual, of having semi-colons where they do not belong. As an example, consider the following fragment.

```
for (int i = 1; i <= 100; i++);  
    System.out.println(i + " " + i*i);
```

This will *not* print a table of squares of the integers from 1 to 100. The body of the `for` statement contains only an empty statement (terminated by the semi-colon on the first line). The `println` statement is not part of the loop at all.

5. Since `while` and `if` statements can have similar forms, it is not uncommon for beginning programmers to use the wrong one. A `while` statement is a type of loop; an `if` statement is not.
6. If you write a `for` statement that starts with the header

```
for (int i = 1; i <= 100; i++)
```

then anybody reading your program expects that the loop will be executed 100 times. If the loop contains a statement of the form

```
if (x < 0)  
    i = 101;
```

then the loop may not be executed 100 times. The header of your loop is, in this case, misleading to a reader of your program. If you were to write something like

```
for (int i = 1; i <= 100 && x >= 0; i++)
```

then your intentions would be much clearer and less likely to cause an error.
7. We saw in Section 4.4 that in `for` statements, the expressions contained in parentheses (following the word `for`) can take many forms. Although there are occasions when these constructs are useful, they can be confusing and should, in general, be avoided. It is almost always better to keep things simple and clear.

Debugging

1. If a program containing a loop compiles correctly but, when it is run, the computer appears to do nothing, then you may have written an infinite loop. Once you have stopped the program, try running it again with tracing statements in the body of the loop. If the program is caught in an infinite loop, any output in the body of the loop will fly by on the screen. To make the tracing statements readable in such circumstances, insert a statement like

```
char junk = In.getChar();
```

after your tracing statement. This will give you a chance to read the output of the tracing statement each time the program executes the body of the loop.
2. It is *very* easy to write a loop that performs one too many or one too few iterations. The difference between failure and success of a

program could be something as simple as the difference between the expressions `while (x > 0)` and `while (x >= 0)`.

3. If a program containing a conditional loop is not performing correctly and the expression that controls the loop is a compound expression, check that all **and**'s, **or**'s, and **not**'s are being used correctly.
4. Using floating point values to control a loop can lead to problems because of roundoff errors. As an example, consider the statement

```
for (double x = 0; x < 1; x += 0.1)
    System.out.println(x);
```

The intention here is that the loop should execute ten times, for `x` having the values 0.0, 0.1, 0.2, ..., 0.9 and should stop after that. Unfortunately, this does not occur. Java does not store floating point values in decimal form. Consequently, values that have an exact decimal form (like 0.1), are often stored as approximations by Java. When `x` “should” have the value 1.0 (stopping the loop), it is actually approximately equal to 0.999 999 999 999. Since this is less than 1.0, the loop is executed an eleventh time. The problem here could be solved by using an integer counter for the loop, as follows.

```
for (int i = 0; i < 10; i++)
    System.out.println(i/10.0);
```

Exercises 4.7

1. In the section on avoiding errors, we stated that the fragment


```
for (int i = 1; i < 100; i++);
    System.out.println(i + " " + i*i);
```

 would *not* print a table of squares of the integers from 1 to 100. What would it print?
2. What value(s) of the variable **response** will stop the following loops?
 - (a) `while (response <= 'a' && response >= 'z') ...`
 - (b) `while (response >= 'A' || response <= 'E') ...`
3. (a) What is wrong with the following fragment?

```

do
{
    System.out.println("Enter transaction code");
    char transCode = In.getChar();
    System.out.println("Code entered: " + transCode
        + "\nIs this correct? (Y/N)");
    char response = In.getChar();
}
while (response != 'Y' || response != 'N');

```

- (b) Modify the fragment so that it behaves in a more appropriate way.

4. How many times will the following loop be executed? Justify your answer.

```

int n = 40;
while (n > 0);
{
    System.out.println(n);
    n /= 2;
}

```

5. Rewrite each loop to make it clearer.

- (a)

```
for (int i = 1; i <= 20; i++)
    if (i % 2 == 0)
        System.out.println(i + " " + i*i);
```
- (b)

```
for (int i = 0; i <= 10; i++)
    if (i*i < 2*i + 4)
        System.out.println(i + " " + i*i);
    else
        i = 11;
```
- (c)

```
int i = 20;
while (i > 0)
{
    System.out.println(i + " " + i*i);
    i--;
}
```

4.8 Review Exercises 4

1. What will be printed by each fragment?

- (a)

```
for (i = 0; i <= 5; i++)  
    System.out.println(i*i);
```
- (b)

```
n = 3;  
do  
    System.out.println(++n);  
while (n < 8);
```
- (c)

```
for (i = 25; i > 0; i -= 4)  
    System.out.println(i);
```
- (d)

```
for(i = j = 1; i < 5; i++)  
{  
    j *= i;  
    System.out.println(j);  
}
```
- (e)

```
n = 26;  
while (n != 1)  
{  
    if (n % 2 == 0)  
        n /= 2;  
    else  
        n = 3 * n + 1;  
    System.out.println(n);  
}
```

2. Describe in a few words what would be stored in the variable `value` after execution of each fragment.

- (a)

```
for (i = value = 0; i <= 10; i++)  
    value += i;
```
- (b)

```
n = In.getInt();  
value = 0;  
while (n % 2 == 0)  
{
```

```

        value++;
        n /= 2;
    }
(c) n = In.getInt();
    value = 0;
    do
    {
        value += n % 10;
        n /= 10;
    }
    while (n > 0);

```

3. A *prime number* is a positive integer that has exactly two distinct divisors — one and the number itself. For example, 17 is a prime number because it is divisible only by 1 and 17. Write a program that reads an integer and then prints a message stating whether or not it is a prime number.
4. The number 153 has the property that it is equal to the sum of the cubes of its digits: $1^3 + 5^3 + 3^3 = 153$. Write a program that will find all the three-digit natural numbers that have this property.
5. Write a program that prompts the user for a natural number and, once one has been supplied, finds and prints all of its exact divisors.
6. The number 6 is said to be a *perfect number* because it is equal to the sum of all its exact divisors (other than itself).

$$6 = 1 + 2 + 3$$

Write a program that finds and prints the three smallest perfect numbers.

7. (a) Write a program that could be used to play a simple game. The program should first ask one user to supply a positive integer less than 1000. Once this user has provided an appropriate value, the program should then ask a second user to guess the number that was given by the first user. After each incorrect guess, the program should tell this user whether the guess was too low or too high and ask for another guess. This should continue until the second user has found the correct number. The program should then print the number of guesses that the second user needed to determine the number.

(b) What guessing strategy will minimize the expected number of incorrect guesses?

8. Let n be a positive integer consisting of the digits $d_k \dots d_1 d_0$. Write a program that reads a value of n and then prints its digits in a column, starting with d_0 . For example, if $n = 3467$, then the program should print

7
6
4
3

9. Rewrite the program in the previous question so that it prints the digits of n in the opposite order, starting with d_k and working down to d_0 .
10. Write a program that takes as input the number of days in a month and the day of the week on which the first of the month occurs. The program should produce a display of the calendar for that month in standard form. For example, if there are 30 days in the month and the first day of the month is Wednesday, the input would be 30 and *Wednesday*, while the output should be similar to the following.

Sun	Mon	Tue	Wed	Thu	Fri	Sat
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30		

11. Write a program that computes the amount to which periodic deposits of \$1 will grow at a given rate of interest and for a given length

of time. The program should prompt the user to provide an interest rate as a percent (from 0 to 20) and a number of compounding periods (from 0 to 50). The program should then produce a table with appropriate headings showing the amount to which \$1 per period (paid at the start of each period) has grown for $1, 2, 3, \dots, n$ periods where n is the value provided by the user. The program should reject invalid input, continuing to prompt the user until valid data have been provided. (Do not attempt to screen for non-numerical input.) Amounts in the table should be rounded to the nearest cent.

12. Write a program that first prompts the user for a value of x , reads x (as a `double`) and then computes and prints the value of e^x without using either of the methods `exp` or `pow` from the `Math` class. The value of e^x is given by the following *power series*.

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

In computing the value of e^x , your program should continue to add terms of the power series until it reaches a term whose absolute value is less than 10^{-15} times the absolute value of the sum of the previous terms.

The symbol $n!$ is read as *n factorial*. If n is a positive integer, then

$$n! = n \times (n - 1) \times (n - 2) \times \dots \times 3 \times 2 \times 1$$

13. Write a program that reads a positive integer and then checks to see whether or not the integer is divisible by 11. For the divisibility test, use the following algorithm, based on one given by the mathematician Charles S. Dodgson (also known as Lewis Carroll).
 - As long as the number has more than one digit, shorten it by deleting the units digit and subtracting this digit from the resulting number.
 - The original number is divisible by 11 if and only if the final number is equal to zero.

You may assume that the input is valid and that it can be stored as a `long` value. Output from the program should consist of the original number followed by any shortened numbers obtained by applying the algorithm followed, finally, by a message stating whether or not the original number was divisible by 11.

As an example, input of 48070 should produce output like the following:

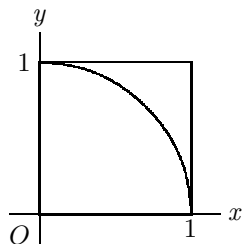
```
48070
4807
473
44
0
48070 is divisible by 11
```

Projects

14. In the nine-digit Social Insurance Number (SIN) given to each person having a job or filing an income tax return in Canada, the ninth digit is a *check digit* that is used to test the validity of the other digits in the SIN. The ninth digit is determined by the following procedure.
 - (a) Double the 2nd, 4th, 6th, and 8th digits.
 - (b) Add the *digits* of the numbers found in step (a).
 - (c) Add the 1st, 3rd, 5th, and 7th digits.
 - (d) Add the numbers found in steps (b) and (c).
 - (e) Subtract the units digit of the result of step (d) from 10 and note the units digit of the result. For the SIN to be valid, its ninth digit must have this value.

Write a program that repeatedly reads nine-digit numbers and determines whether or not each number is a valid SIN. The program should stop when it reads the value 999999999.

15. The diagram shows a unit square in the Cartesian plane and a quarter circle of radius one, centred at the origin.



- (a) Show that, if a point is chosen at random within the square, the probability that it will also be inside the quarter circle is $\pi/4$.
 - (b) Write a program that repeatedly generates the coordinates of a random point within the square and determines whether or not the point also lies inside the quarter circle. The program should repeat the process 10 000 times. After every 1 000 repetitions, it should use the results to determine and print an estimate of the value of π (based on all repetitions up to that point).
16. Assuming that there are 365 days in a year and that the probability that a person will be born on a given day is $1/365$, we can calculate the probability that, in a group of five people chosen at random, at least two will have the same birthday, as follows:

We start by calculating the probability that all five have different birthdays. This will be

$$\frac{365}{365} \times \frac{364}{365} \times \frac{363}{365} \times \frac{362}{365} \times \frac{361}{365}$$

The reasoning here is that the first person could be born on any of 365 days; the second person, to have a different birthday, could only be born on 364 out of 365 days; the third person, to have a birthday different from either of the first two, could only be born on 363 out of 365 days; and so on to the fifth person. Now, to find the probability that at least two people in a randomly chosen group of five have the same birthday, we simply subtract the above expression from 1 to obtain

$$1 - \frac{365}{365} \times \frac{364}{365} \times \frac{363}{365} \times \frac{362}{365} \times \frac{361}{365} \doteq 0.027$$

Write a program that produces a table showing the minimum number of people that would be required so that the probability of at least two people in a randomly chosen group having the same birthday is at least 0.1, 0.2, ..., 0.9, 1.0

17. Write a program that will determine the roots of equations of the form $ax^2 + bx + c = 0$. The program should repeatedly prompt the user for values of a , b , and c . For each set of values, the program should solve the corresponding equation, if it has a solution, or print an appropriate message, if it has no solution. The program should be able to cope with coefficients that produce either real or complex

roots. It should also be able to give appropriate results if one or more of the coefficients are equal to zero. The program should be interactive. The form of the exchange between a user and a computer is shown in the following example.

```
Do you want to solve an equation (y/n)? y
Enter a: 1
Enter b: -5
Enter c: 6
Root1: 2.0
Root2: 3.0
```

```
Do you want to solve an equation (y/n)? y
Enter a: 1
Enter b: -1
Enter c: 4.25
Root1: real part: 0.5 imaginary part: -2.0
Root2: real part: 0.5 imaginary part: 2.0
```

```
Do you want to solve an equation (y/n)? y
Enter a: 0
Enter b: 2.4
Enter c: 9.0
Root1: -3.75
```

```
Do you want to solve an equation (y/n)? n
```

18. Write a program that will perform the four basic arithmetic operations (addition, subtraction, multiplication, and division) on pairs of fractions, producing answers in standard fractional form. Specifically, your program should repeatedly perform the following actions.
 - Read a character representing an operation. This will be one of the characters +, -, *, /, or \$. The \$ signals the end of the data; reading it should cause the program to terminate.
 - Read four integers representing the numerator and denominator of a first fraction followed by the numerator and denominator of a second fraction.
 - Perform the indicated operation and print the results in the appropriate form as illustrated in the examples that follow.

In the examples, the input is shown on one line. The program, however, should read the input items one at a time, with each one on a separate line.

- (a) Input: + 1 3 -19 -8
Output: $1/3 + -19/-8 = 2$ and $17/24$
- (b) Input: / 2 -5 3 -7
Output: $2/-5 / 3/-7 = 14/15$
- (c) Input: - 5 2 1 2
Output: $5/2 - 1/2 = 2$

Notice that improper fractions should be written as mixed numbers, as in (a), and integer results should be written as such, as in (c). It is not, however, necessary to reduce fractions to lowest terms. The program should assume that a user will provide integers when required but it should not assume that the integers will produce valid results. For example, bad data such as

+ 3 5 4 0

should be detected by the program. In such cases, the program should write an error message and then continue to process the next set of data.

19. The *Russian Peasant Algorithm* is a process for determining the product of two positive integers using only multiplication by two, division by two, and addition. The Russian Peasant Algorithm can be stated as follows:
 - Write each number (the multiplier and the multiplicand) at the head of a column.
 - Double the number in the first column, and halve the number in the second column.
 - If the number in the second column is odd, ignore the remainder when dividing.
 - If the number in the second column is even, cross out that entire row after dividing.
 - Keep doubling, halving, and crossing out until the number in the second column is 1.
 - Add up the remaining numbers in the first column.
 - The total is the product of your original numbers.

Here is an example, determining the product $57 \times 86 = 4902$:

```

57 86
114 43
228 21
456 10
912 5
1824 2
3648 1
4902

```

Write a program that prompts the user for a multiplier and a multiplicand and then uses the Russian Peasant Algorithm to determine the product. Your program must also show the intermediate steps the algorithm takes by printing out any row where an addition is required in the first column. Your program should continue prompting the user for input until two zeros are entered. This should terminate the program. If a user provides negative values for either the multiplier or the multiplicand, these should be rejected. If one value is zero and the other is positive, the program should give the correct result without using the algorithm. Using the example above, your program should function as shown in the following sample:

```

Multiplier: 57
Multiplicand: 86
Calculating product:
114 43
228 21
912 5
3648 1
Product: 4902
Multiplier: 48
Multiplicand: -36
Values must not be negative
Multiplier: 27
Multiplicand: 0
Product: 0
Multiplier: 0
Multiplicand: 0

```